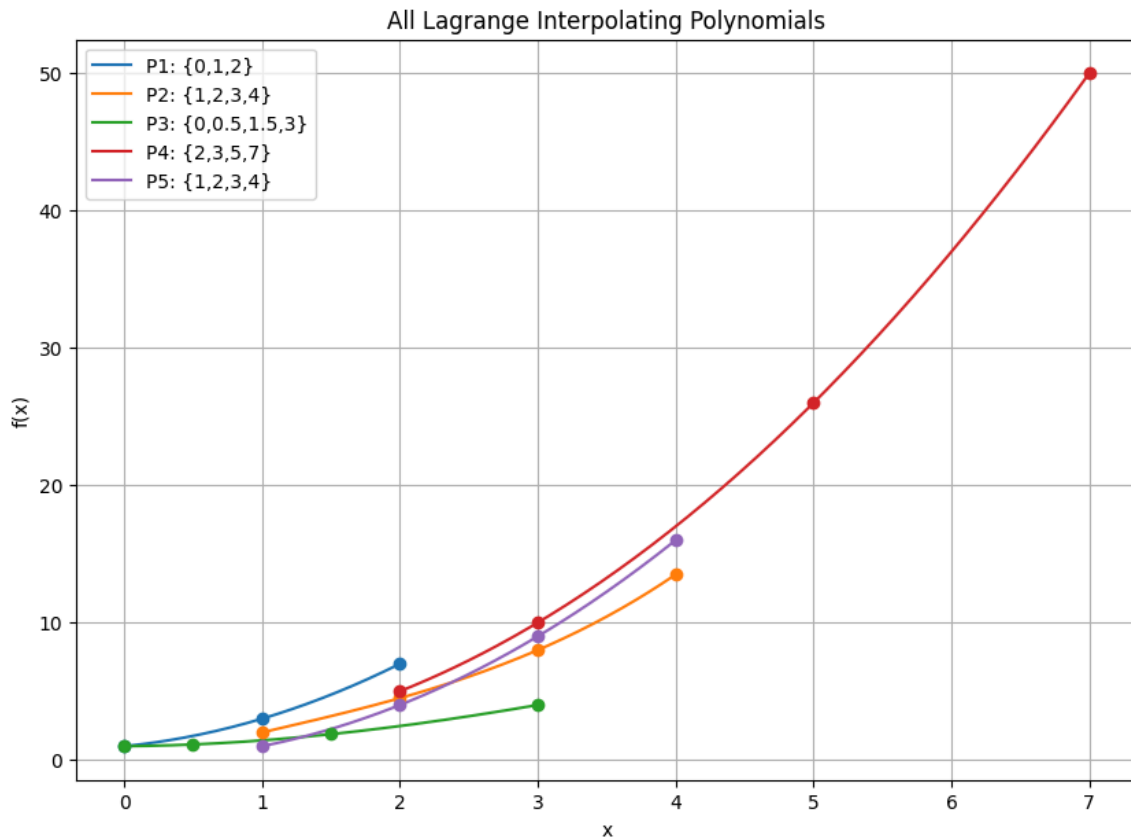


Numerical Computing II — Assignment IV: Lagrange Interpolation

Name: Saily Kadam

Roll no: MS 2511



Lagrange Interpolation Algorithm

FUNCTION lagrange_polynomial(x_data, y_data):

$n = \text{length}(x_data)$

$P(x) = 0$

$\text{basis_list} = []$

FOR i in 0 to $n-1$:

$L_i(x) = 1$

FOR j in 0 to $n-1$:

IF $j \neq i$:

```

    L_i(x) = L_i(x) * (x - x_data[j]) / (x_data[i] - x_data[j])
    basis_list.append(L_i)
    P(x) = P(x) + y_data[i] * L_i(x)
    RETURN expand(P(x), basis_list)

---
```

Solutions

Problem 1

1. Lagrange Basis Polynomials:

$$L_0(x) = 0.5x^2 - 1.5x + 1,$$

$$L_1(x) = -x^2 + 2x,$$

$$L_2(x) = 0.5x^2 - 0.5x.$$

2. Interpolating Polynomial: $P_2(x) = x^2 + x + 1$.

3. Verification: $L_0(0)=1$,

$$L_1(1)=1,$$

$$L_2(2)=1,$$

and all other combinations $L_i(x_j)$ are 0.

4. Estimate $f(1.5)$: $P_2(1.5) = 4.75$.

5. Comparison: The interpolated value 4.75 exactly matches the exact function value $f(1.5) = 4.75$.

Problem 2

1. Interpolating Polynomial: $P_3(x) = (1/6)x^3 - 0.5x^2 + (17/6)x - 0.5$.

2. Estimate $f(2.5)$: $P_3(2.5) = 6.0625$.

3. Estimate $f(3.5)$: $P_3(3.5) = 10.4375$.

4. Reasonableness: Both estimates fall within the range $[1, 4]$ and follow the accelerating trend of the data.

Problem 3

1. Interpolating Polynomial: $P_3(x) = -0.02222x^3 + 0.37778x^2 + 0.06667x + 1$.
2. Estimate $f(1)$: $P_3(1) = 1.4222$.
3. Estimate $f(2)$: $P_3(2) = 2.4667$.
4. Convenience: Lagrange interpolation handles non-uniform x-spacing directly without needing to compute complex finite-difference tables.

Problem 4

1. Construct $P_3(x)$: $P_3(x) = x^2 + 1$.
2. Calculate $f(4)=17$, $f(6)=37$, $f(8)=65$.
3. Reliability: $f(4)$ and $f(6)$ are most reliable as they are interpolations within the range $[2, 7]$, while $f(8)$ is an extrapolation.

Problem 5

1. Correctness: The statement is false; the polynomial degree is at most $n-1$, not necessarily exactly $n-1$.
2. Construct $P(x)$: $P(x) = x^2$.
3. Degree: The degree of the resulting polynomial is 2.
4. Degree logic: If the underlying data points lie on a lower-degree curve, higher-order coefficients in the Lagrange formula cancel out to zero.

to lower-degree function, e.g. data actually satisfies $f(x) = x^2$

```
[8]: x_data5 = [1, 2, 3, 4]
     y_data5 = [1, 4, 9, 16]

     P5, basis5 = lagrange_polynomial(x_data5, y_data5)
     print("P(x) =", P5)
     print("Polynomial degree =", sp.degree(P5, x))
     print("\nStudent's claim is false: n data points guarantee a degree of AT MOST n-1.")
     print("If the underlying data points lie on a lower-degree curve (Like f(x) = x^2),")
     print("higher-order terms algebraically cancel out during expansion.")
```

```
P(x) = x**2
Polynomial degree = 2
```

Student's claim is false: n data points guarantee a degree of AT MOST n-1.
If the underlying data points lie on a lower-degree curve (Like $f(x) = x^2$),
higher-order terms algebraically cancel out during expansion.

```
[7]: x_data4 = [2, 3, 5, 7]
     y_data4 = [5, 10, 26, 50]

     P4, basis4 = lagrange_polynomial(x_data4, y_data4)
     print("P3(x) =", P4)

     f4 = P4.subs(x, 4)
     f6 = P4.subs(x, 6)
     f8 = P4.subs(x, 8)
     print(f"\nf(4) = {sp.N(f4)}")
     print(f"\nf(6) = {sp.N(f6)}")
     print(f"\nf(8) = {sp.N(f8)}")

     print("\nx=4 and x=6 lie inside [2,7] -> interpolation -> more reliable.")
     print("\nx=8 lies outside [2,7] -> extrapolation -> least reliable of the three.")
```

```
P3(x) = x**2 + 1
```

```
f(4) = 17.0000000000000
f(6) = 37.0000000000000
f(8) = 65.0000000000000
```

x=4 and x=6 lie inside [2,7] -> interpolation -> more reliable.
x=8 lies outside [2,7] -> extrapolation -> least reliable of the three.

```
[6]: x_data3 = [0, 0.5, 1.5, 3]
     y_data3 = [1.000, 1.125, 1.875, 4.000]

     P3, basis3 = lagrange_polynomial(x_data3, y_data3)
     print("P(x) =", P3)

     est_1 = P3.subs(x, 1)
     est_2 = P3.subs(x, 2)
     print(f"\nP(1) = {sp.N(est_1)}")
     print(f"\nP(2) = {sp.N(est_2)}")

     print("\nLagrange interpolation builds basis polynomials directly from the")
     print("x_i values (no assumption of equal spacing), so it handles this")
     print("non-uniform grid natively - unlike finite-difference-based methods.")

     P(x) = -0.022222222222221*x**3 + 0.377777777777777*x**2 + 0.0666666666666673*x + 1.0

     P(1) = 1.42222222222222
     P(2) = 2.46666666666667
```

Lagrange interpolation builds basis polynomials directly from the
 x_i values (no assumption of equal spacing), so it handles this
non-uniform grid natively - unlike finite-difference-based methods.

```
x_data = [2, 3, 5, 7] y_data = [5, 10, 26, 50]
```

```
[4]: x_data1 = [0, 1, 2]
     y_data1 = [1, 3, 7]

     P1, basis1 = lagrange_polynomial(x_data1, y_data1)
     print("P2(x) =", P1)

     for i, Li in enumerate(basis1):
         print(f"L_{i}(x) =", sp.expand(Li))

     verify_basis(basis1, x_data1)

     est_1_5 = P1.subs(x, 1.5)
     exact_1_5 = 1.5**2 + 1.5 + 1
     print(f"\nP2(1.5) = {est_1_5}")
     print(f"\nExact f(1.5) = x^2+x+1 -> {exact_1_5}")
     print(f"\nError = {abs(float(est_1_5) - exact_1_5)}")

     P2(x) = x**2 + x + 1
     L_0(x) = x**2/2 - 3*x/2 + 1
     L_1(x) = -x**2 + 2*x
     L_2(x) = x**2/2 - x/2
     Verification of L_i(x_j) = delta_ij:
     L_0(x_j) for j=0..2: [1, 0, 0]
     L_1(x_j) for j=0..2: [0, 1, 0]
     L_2(x_j) for j=0..2: [0, 0, 1]

     P2(1.5) = 4.75000000000000
     Exact f(1.5) = x^2+x+1 -> 4.75
     Error = 0.0

     x_data = [1, 2, 3, 4] y_data = [2.0, 4.5, 8.0, 13.5]
```